

ISRM Lab 2: CI/CD Pipeline & Automated Security Scanning

Course: Information Security & Risk Management (ISRM)

Focus: Jenkins, GitHub, Bandit, DevSecOps Security Gates

LAB OBJECTIVE

The objective of this lab is to implement a complete **CI/CD Security Pipeline** using Jenkins and GitHub for the Student Management System developed in Lab 1.

This phase includes:

1. Creating GitHub branches for vulnerable and fixed versions
2. Configuring Jenkins pipeline automation
3. Integrating Bandit security scanner
4. Implementing Security Gates
5. Comparing vulnerable and fixed pipeline results

PART A: GITHUB BRANCH IMPLEMENTATION

1. GitHub Repository Setup

Commands Used:

```
git init
git add .
git commit -m "Initial_vulnerable_version"
git branch vulnerable-version
git checkout -b fixed-version
```

Branch Structure:

Branch Name	Purpose
vulnerable-version	Contains insecure Student Management System
fixed-version	Contains remediated secure implementation

Repository Structure:

```
ISRM_Proj/  
|  
|--- Jenkinsfile  
|--- app.py  
|--- database.py  
|--- config.py  
|--- requirements.txt  
|--- reports/  
|--- generate_vulnerability_report.py
```

PART B: JENKINS INSTALLATION & SETUP

1. Jenkins Installation

Installation Steps:

1. Download Jenkins from: <https://www.jenkins.io/download/>
2. Install Jenkins using Windows Installer (.msi)
3. Start Jenkins service on Port 8080
4. Open browser:

```
http://localhost:8080
```

2. Unlock Jenkins

Windows Command:

```
type C:\ProgramData\Jenkins\.jenkins\  
secrets\initialAdminPassword
```

3. Plugin Installation

Installed plugins:

- Git Plugin
- Pipeline Plugin
- GitHub Integration Plugin
- HTML Publisher Plugin

PART C: VULNERABLE PIPELINE IMPLEMENTATION

1. Vulnerable Jenkinsfile

Pipeline Code:

```
pipeline {
    agent any

    environment {
        PYTHON_PATH = "C:\\Python39\\python.exe"
        REPORT_DIR = "reports"
    }

    triggers {
        githubPush()
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                bat """
                python -m venv venv
                call venv\\Scripts\\activate
                pip install -r requirements.txt
                pip install bandit
                """
            }
        }

        stage('Security_Scan') {
            steps {
```

```

        bat """
        bandit-r-f.json
        -o reports\\bandit_report.json
        """
    }
}

stage('SecurityGate') {

    steps {

        script {

            def report =
            readFile('reports/bandit_report.json')

            if(report.contains('HIGH')) {

                currentBuild.result = 'FAILURE'
            }
        }
    }
}
}

```

2. Vulnerabilities Detected by Bandit

Issue	Severity	Reason
SQL Injection	HIGH	Dynamic SQL query construction
Hardcoded Password	LOW	Secrets inside source code
debug=True	HIGH	Flask running in debug mode
Bind All Interfaces	MEDIUM	Application exposed publicly
Weak File Upload	HIGH	No validation on uploaded files

PART D: FIXED PIPELINE IMPLEMENTATION

1. Fixed Jenkinsfile

Secure Pipeline Code:

```

pipeline {
    agent any

```

```

environment {

    PYTHON_PATH = "C:\\Python39\\python.exe"

    REPORT_DIR = "reports"
}

triggers {

    githubPush()
}

stages {

    stage('Checkout') {

        steps {

            cleanWs()

            checkout scm
        }
    }

    stage('Build') {

        steps {

            bat """

python -m venv venv

call venv\\Scripts\\activate

pip install --upgrade pip

pip install -r requirements.txt

pip install bandit
"""

        }
    }

    stage('Security_Scan') {

        steps {

            catchError(
                buildResult: 'SUCCESS',
                stageResult: 'FAILURE') {

                bat """

```

```

#####if_not_exist_reports_mkdir_reports

#####bandit_r_._f_json
#####-o_reports\\bandit_report.json

#####bandit_r_._f_html
#####-o_reports\\bandit_report.html
#####""
        }
    }
}

stage('Archive_Reports') {

    steps {

        archiveArtifacts artifacts:
        'reports/**',
        allowEmptyArchive: true
    }
}

post {

    success {

        echo "Secure_Build_Passed"
    }

    failure {

        echo "Security_Vulnerabilities_Detected"
    }
}
}

```

2. Security Improvements

- Workspace cleanup implemented
- Security reports archived
- HTML + JSON reports generated
- Improved build handling
- Security Gate enforced

PART E: GITHUB WEBHOOK IMPLEMENTATION

1. Webhook Configuration

GitHub Webhook Settings:

Setting	Value
Payload URL	http://localhost:8080/github-webhook/
Content Type	application/json
Event Type	Push Event
Active	Enabled

2. Automatic Trigger Test

Commands Used:

```
git add .  
  
git commit -m "Testing_webhook_trigger"  
  
git push origin main
```

Result:

- GitHub webhook triggered automatically
- Jenkins pipeline started automatically
- Security scan executed successfully

PART F: PIPELINE EXECUTION RESULTS

1. Vulnerable Version Output

```
===== SECURITY SCANNING =====  
  
Issue: SQL Injection  
Severity: HIGH  
  
Issue: debug=True  
Severity: HIGH  
  
Issue: Hardcoded Password  
Severity: LOW  
  
Build FAILED
```

2. Fixed Version Output

```
===== SECURITY SCANNING =====  
  
No HIGH severity vulnerabilities found  
  
Build PASSED
```

PART G: PIPELINE COMPARISON

Pipeline Stage	Vulnerable Version	Fixed Version
Checkout	Success	Success
Build	Success	Success
Security Scan	HIGH Issues Found	No HIGH Issues
Security Gate	FAILED	PASSED
Report Generation	JSON Only	JSON + HTML

PART H: IMPLEMENTATION SUMMARY

Feature	Tool Used	Purpose
Version Control	GitHub	Source Code Management
CI/CD Pipeline	Jenkins	Automation
Security Scanner	Bandit	Static Security Analysis
Webhook Trigger	GitHub Webhook	Automatic Build Trigger
Security Gate	Jenkins Script	Block Vulnerable Builds

CONCLUSION

In Lab 2, a complete CI/CD security pipeline was implemented using Jenkins, GitHub, and Bandit. Two separate GitHub branches were created to compare vulnerable and secure implementations of the Student Management System.

The vulnerable pipeline intentionally failed due to HIGH severity vulnerabilities detected by Bandit, while the fixed pipeline successfully passed all security checks after remediation.

This lab demonstrated the practical implementation of DevSecOps principles including automated security scanning, security gates, webhook automation, and continuous integration for secure software development.

— End of Report —